BÁO CÁO KẾT QUẢ THỰC TẬP

Tuần 2: Làm Việc Với Database và Hoàn Thiện CRUD

I. LÝ THUYẾT

1. Database và So sánh giữa SQL và NoSQL

1.1. Khái niệm Database

- Database là hệ thống lưu trữ và tổ chức dữ liệu có cấu trúc, giúp người dùng và ứng dụng dễ dàng truy xuất, thêm, sửa, xóa và quản lý thông tin.Database thường được quản lý thông qua hệ quản trị cơ sở dữ liệu như PostgreSQL, MySQL, SQL Server hoặc MongoDB.
- Vai trò của Database:
    + Lưu trữ và quản lý dữ liệu tập trung.
    + Hỗ trợ các thao tác CRUD (Create, Read, Update, Delete).
    + Đảm bảo tính toàn vẹn, nhất quán và bảo mật dữ liệu.
    + Cho phép nhiều người dùng hoặc ứng dụng truy cập dữ liệu.

1.2. SQL và NoSQL

- SQL là ngôn ngữ truy vấn có cấu trúc, được sử dụng để quản lý và thao tác với cơ sở dữ liệu quan hệ. Dữ liệu được tổ chức thành các bảng gồm hàng và cột, có thể liên kết với nhau thông qua khóa chính và khóa ngoại .Ví dụ: PostgreSQL, MySQL, SQL Server, SQLite.
- NoSQL là nhóm hệ quản trị cơ sở dữ liệu phi quan hệ, hỗ trợ nhiều mô hình lưu trữ như document, key-value,... NoSQL thường có cấu trúc dữ liệu linh hoạt, phù hợp với nhiều loại dữ liệu và nhu cầu mở rộng khác nhau.

1.3. Bảng so sánh SQL và NoSQL

Tiêu chí	SQL	NoSQL
Mô hình dữ liệu	Dữ liệu dạng bảng	Document, key-value, graph,..

Cấu trúc dữ liệu	Schema được định nghĩa rõ ràng	Thường linh hoạt hơn
Ngôn ngữ truy vấn	Sử dụng SQL	Tùy thuộc từng hệ quản trị
Ví dụ	PostgreSQL, MySQL	MongoDB

2. Phương thức giao tiếp giữa NestJS với MongoDB, PostgreSQL.

- Quy trình kết nối cơ bản:

1. Cài đặt hệ quản trị cơ sở dữ liệu.
2. Cài đặt thư viện kết nối tương ứng.
3. Cấu hình thông tin kết nối trong file `.env`.
4. Đăng ký kết nối trong NestJS.
5. Xây dựng Service để thao tác dữ liệu và kiểm tra kết nối.

II. NỘI DUNG THỰC HÀNH & CÁC CÂU LỆNH CẤU HÌNH

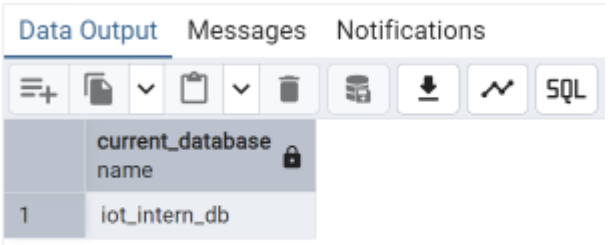
1. Các câu lệnh

- Cài đặt module quản lý biến môi trường toàn cục :npm install @nestjs/config
- Cài đặt định nghĩa kiểu dữ liệu cho môi trường kiểm thử:
  + npm install --save-dev @types/jest
- Chạy toàn bộ hệ thống kiểm thử tự động trên dự án: npm run test
- Khởi động máy chủ phát triển ở chế độ theo dõi thay đổi của file: npm run start:dev
- Cài Prisma CLI để quản lý database và `dotenv` để đọc biến môi trường từ file `.env`:
  + `npm install prisma@7 dotenv --save-dev`
- Cài Prisma Client để truy vấn dữ liệu và các thư viện giúp kết nối với PostgreSQL:
  + `npm install @prisma/client@7 @prisma/adapter-pg pg`
- Cài định nghĩa kiểu dữ liệu TypeScript cho thư viện PostgreSQL:
  + `npm install -D @types/pg`

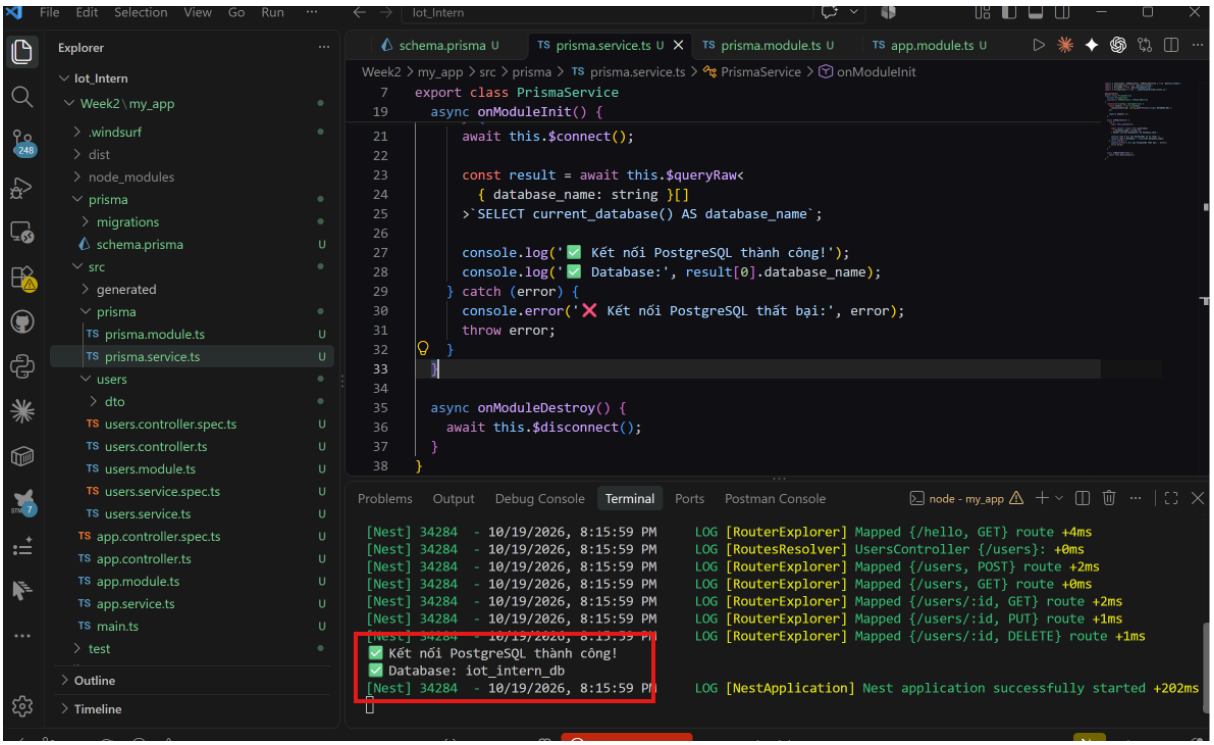
- Tạo hoặc cập nhật các bảng trong PostgreSQL dựa trên `schema.prisma`:
    + npx prisma migrate dev --name init
- Sinh Prisma Client để NestJS thao tác với database:
    + npx prisma generate

2. Nội dung thực hành

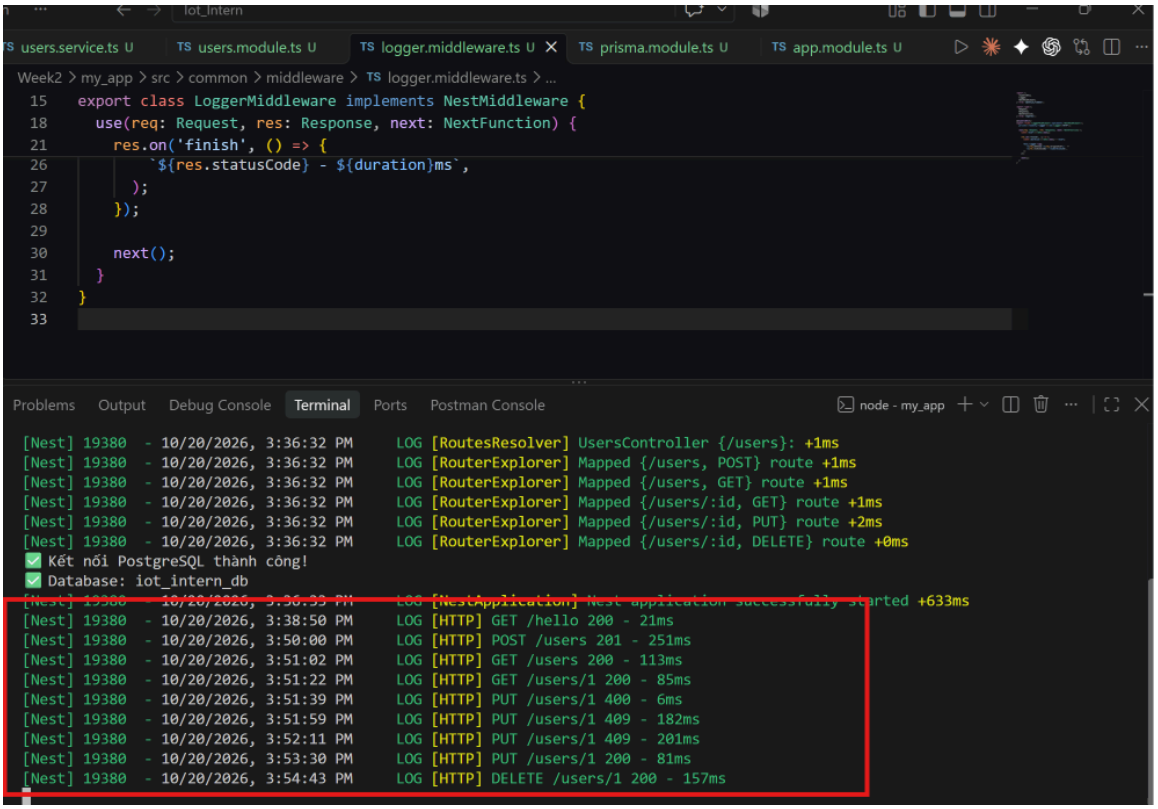
- Cài đặt PostgreSQL/MySQL và tạo database.



- Khởi tạo prisma:
    + npx prisma init --datasource-provider postgresql --output ../src/generated/prisma
- Kết nối NestJS với database:

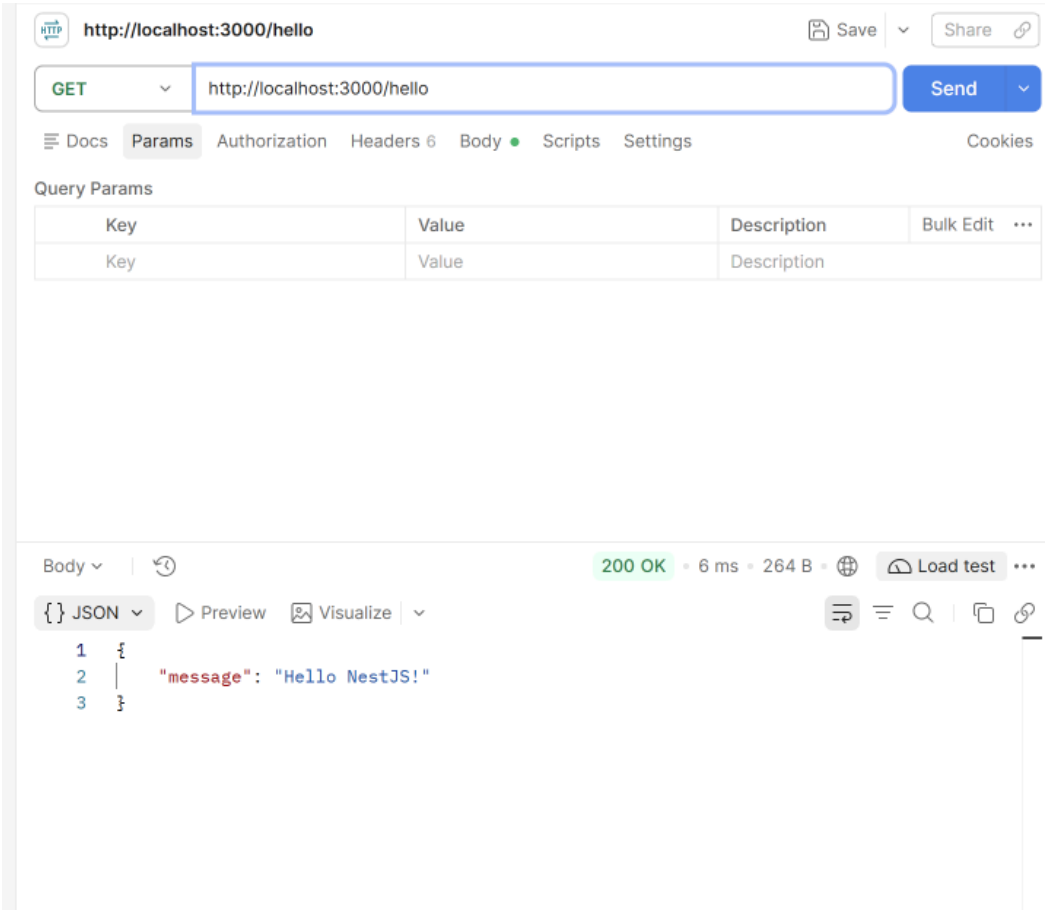


- Viết Middleware để log request:
    + Sau khi kiểm thử các API CRUD bằng Postman, LoggerMiddleware đã ghi nhận thành công phương thức HTTP, đường dẫn, mã trạng thái và thời gian xử lý trên Terminal.



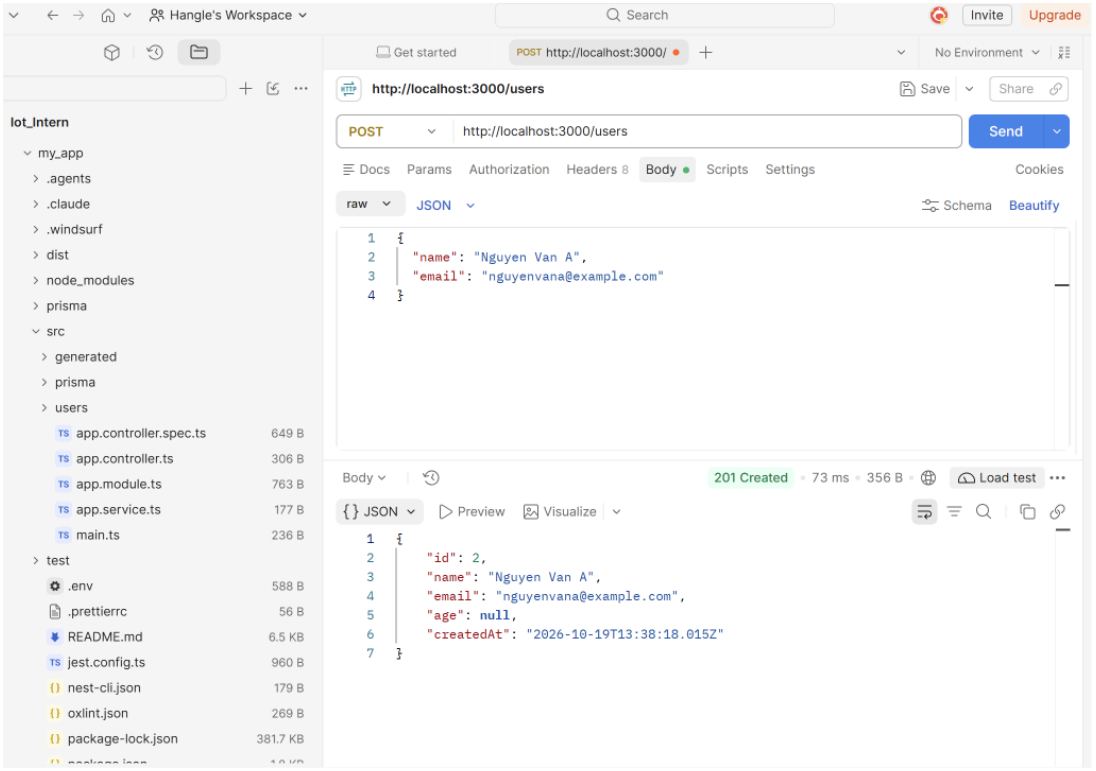
III. KẾT QUẢ KIỂM THỬ

1. Kiểm thử API Health: GET /hello

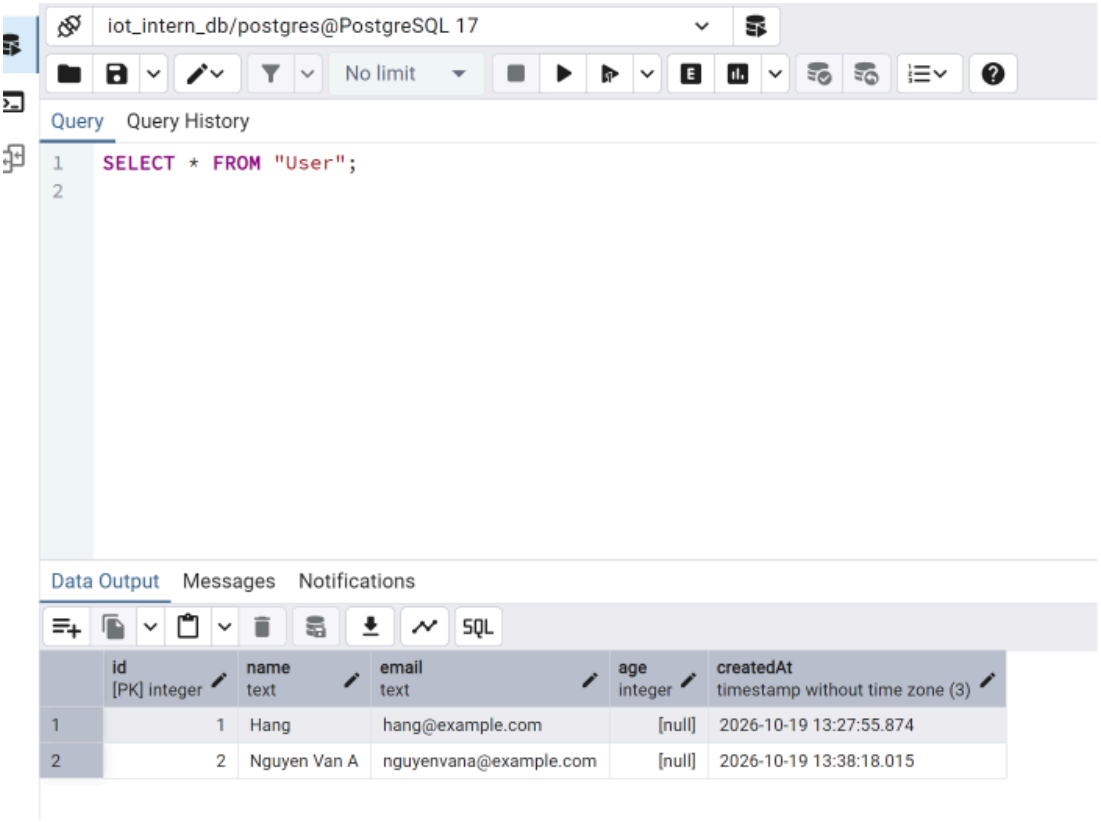


2. Kiểm thử API CRUD Users: POST /users

- Sau khi gửi request POST /users bằng Postman, API trả về kết quả thành công và thông tin người dùng vừa được tạo. Kết quả cho thấy người dùng mới Nguyen Van A đã được thêm vào bảng `User` với ID tự động tăng là 2. Điều này xác nhận API POST đã hoạt động đúng và dữ liệu được lưu thành công vào database `iot_intern_db` thông qua Prisma ORM.

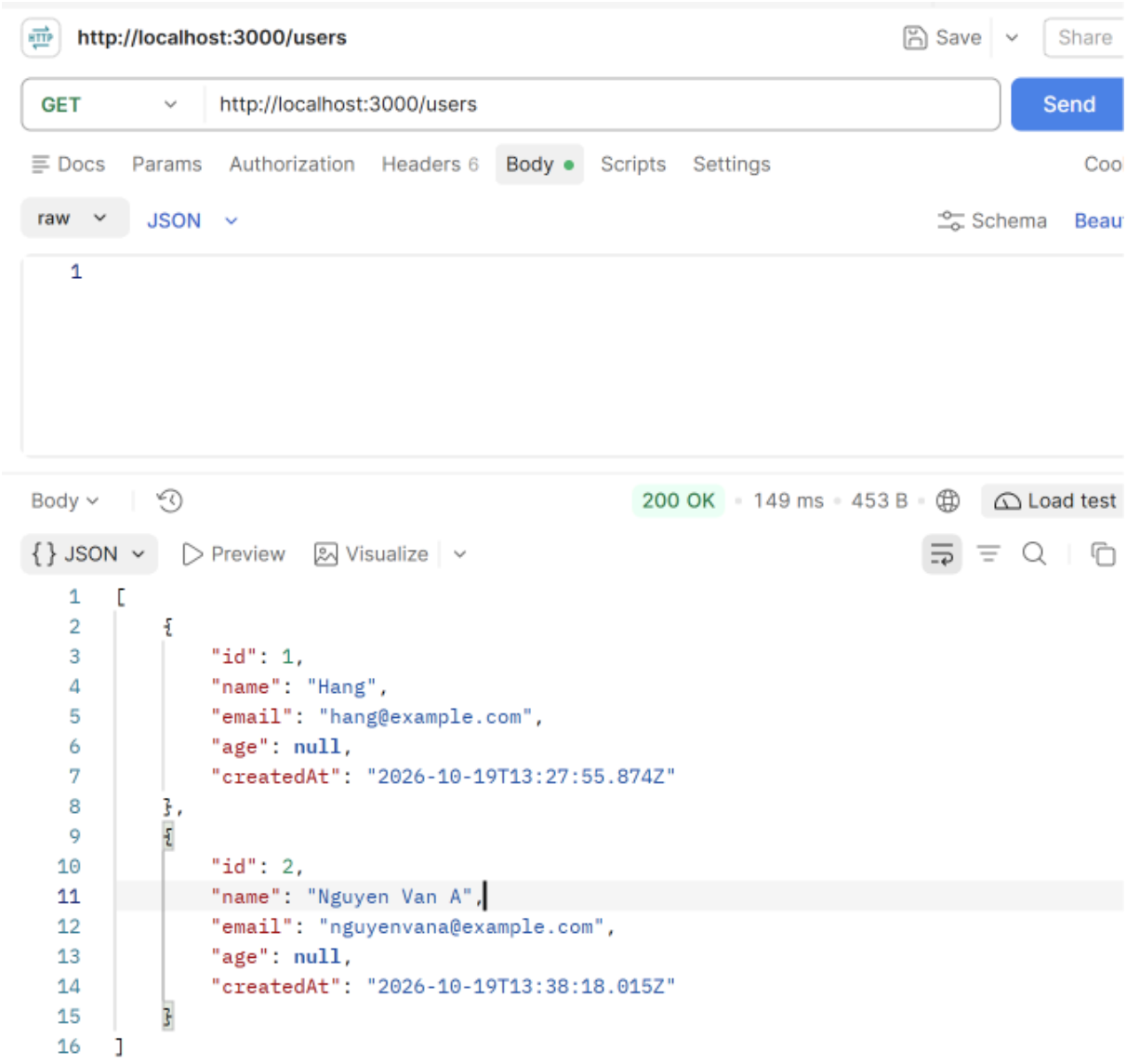

Hangle's Workspace
Search
Invite
Upgrade
Get started
POST http://localhost:3000/
No Environment
http://localhost:3000/users
Save
Share
POST
http://localhost:3000/users
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"name": "Nguyen Van A",
"email": "nguyenvana@example.com"
}
Body
201 Created
73 ms
356 B
Load test
JSON
Preview
Visualize
{
"id": 2,
"name": "Nguyen Van A",
"email": "nguyenvana@example.com",
"age": null,
"createdAt": "2026-10-19T13:38:18.015Z"
}
Iot_Intern
my_app
.agents
.claude
.windsurf
dist
node_modules
prisma
src
generated
prisma
users
app.controller.spec.ts 649 B
app.controller.ts 306 B
app.module.ts 763 B
app.service.ts 177 B
main.ts 236 B
test
.env 588 B
.prettierrc 56 B
README.md 6.5 KB
jest.config.ts 960 B
nest-cli.json 179 B
oxlint.json 269 B
package-lock.json 381.7 KB

Kết quả :


iot_intern_db/postgres@PostgreSQL 17
No limit
Query
Query History
SELECT * FROM "User";
Data Output
Messages
Notifications
SQL
id [PK] integer
name text
email text
age integer
createdAt timestamp without time zone (3)
1 1 Hang hang@example.com [null] 2026-10-19 13:27:55.874
2 2 Nguyen Van A nguyenvana@example.com [null] 2026-10-19 13:38:18.015

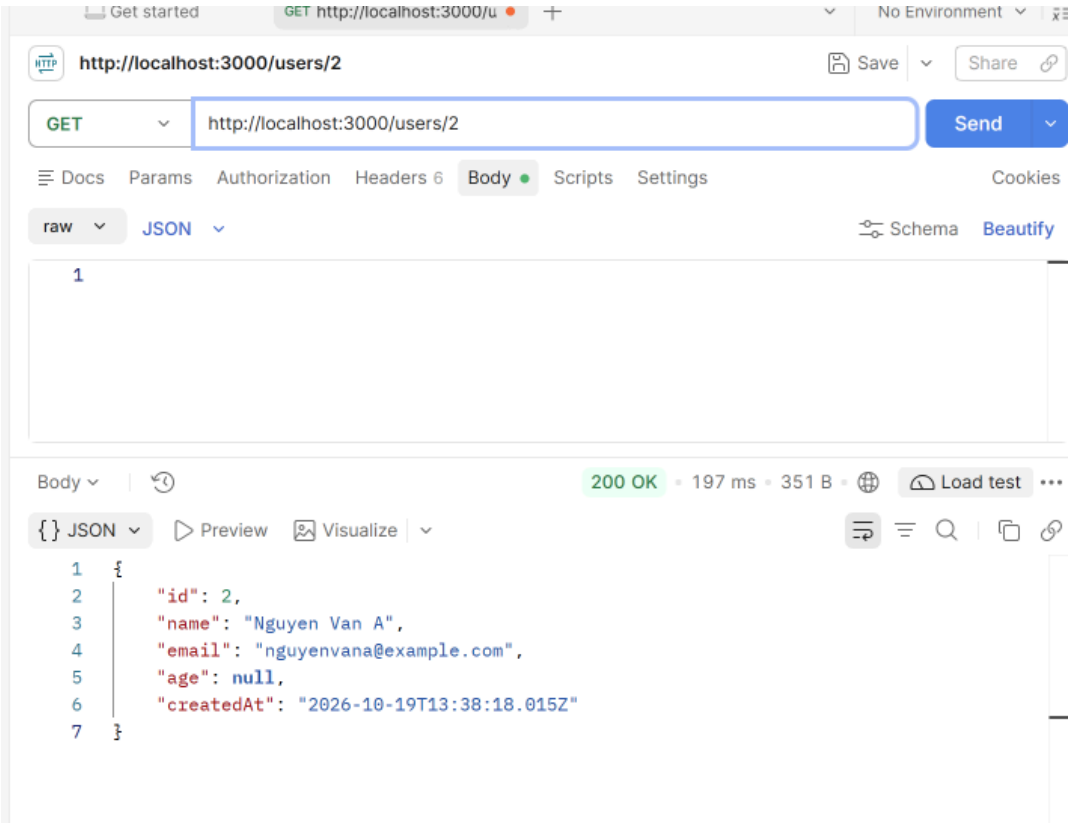
3. **Kiểm thử API CRUD Users: GET /users**

- Sau khi gửi request GET /users bằng Postman, API trả về trạng thái 200 OK cùng danh sách người dùng được truy xuất thành công từ PostgreSQL thông qua Prisma ORM.



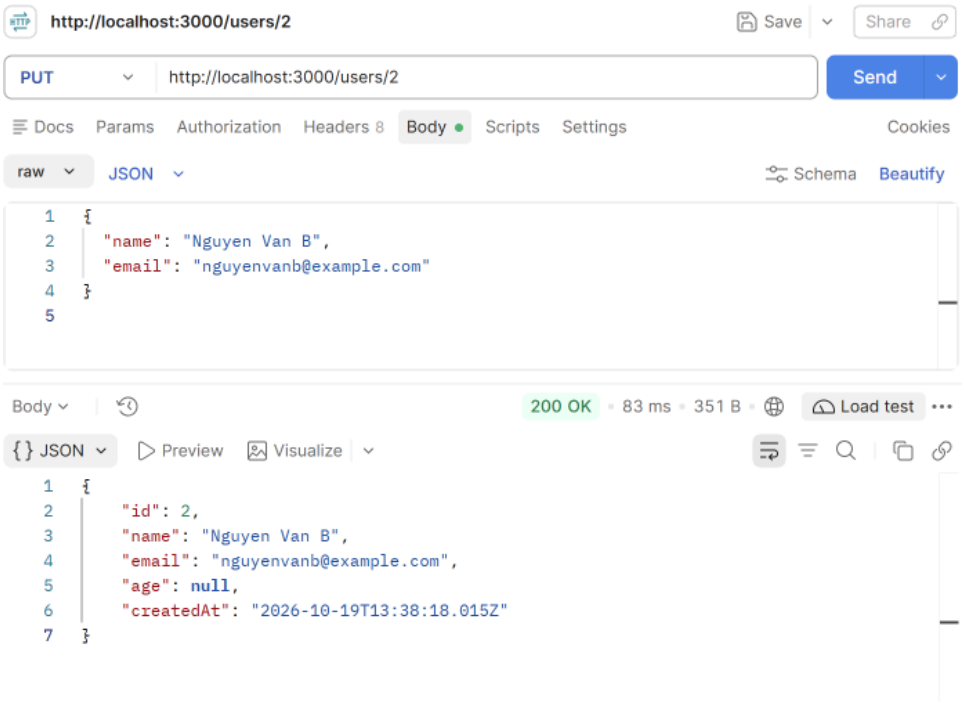
4. **Kiểm thử API CRUD Users: GET /users/:id**

- Sau khi gửi request **GET /users/2** bằng Postman, API trả về trạng thái **200 OK** cùng thông tin người dùng có ID = 2, xác nhận chức năng tìm kiếm người dùng theo ID hoạt động thành công với dữ liệu từ PostgreSQL.

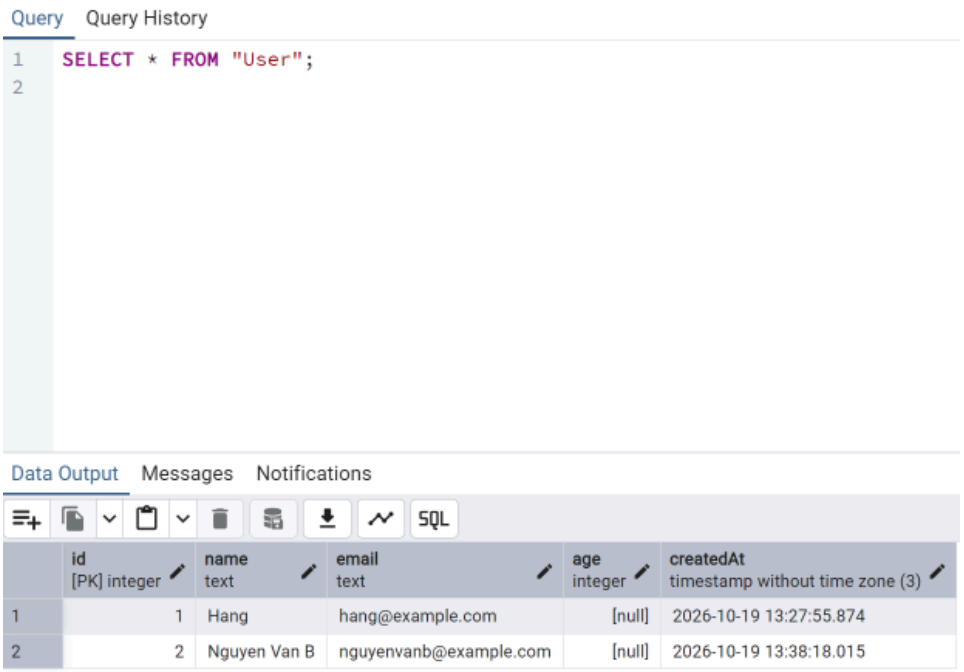


5. **Kiểm thử API CRUD Users: PUT /users/:id**

- Sau khi gửi request **PUT /users/2** bằng Postman, API trả về trạng thái **200 OK** và thông tin người dùng đã được cập nhật thành công trong PostgreSQL thông qua Prisma ORM.

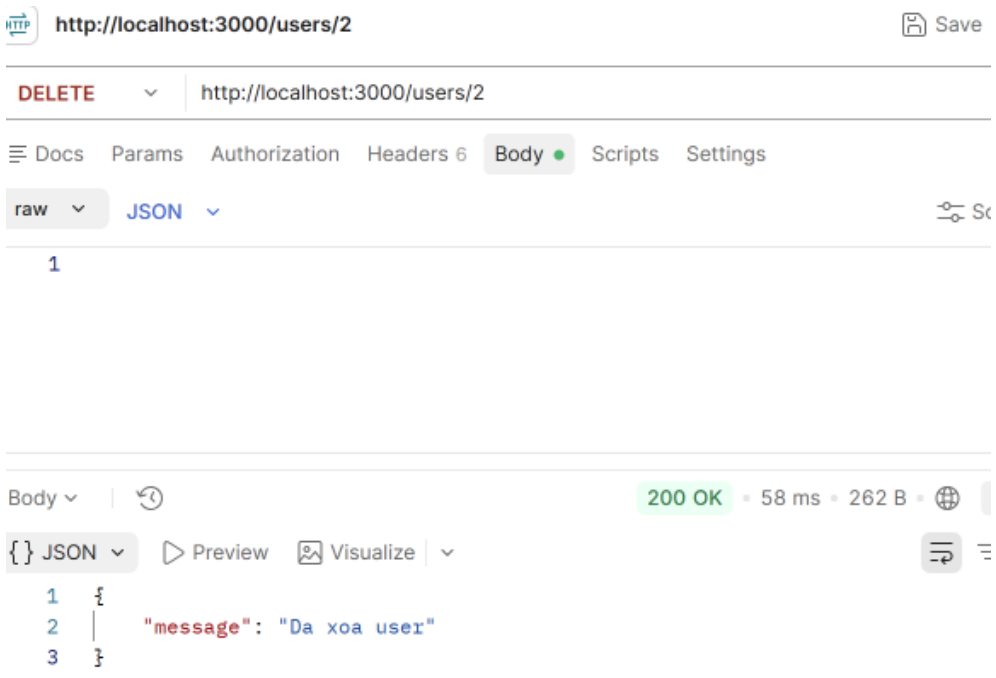


Kết quả:



6. **Kiểm thử API CRUD Users: DELETE /users/:id**

- Sau khi gửi request DELETE /users/2 bằng Postman, API trả về trạng thái 200 OK, xác nhận người dùng có ID = 2 đã được xóa thành công khỏi PostgreSQL thông qua Prisma ORM.



Kết quả:

Query | Query History

```
1  SELECT * FROM "User";
2
```

Data Output | Messages | Notifications

	id [PK] integer	name text	email text	age integer	createdAt timestamp without time zone (3)
1	1	Hang	hang@example.com	[null]	2026-10-19 13:27:55.874